Ders 28 Genişletilmiş Notu: İnternet Uygulamaları, Web Teknolojileri ve Binary Search

Ders 28 Genişletilmiş Notu: İnternet Uygulamaları, Web Teknolojileri ve Binary Search

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. IP Adresleri ve Alan Adları
- 2. İnternet Uygulamaları ve Protokoller
- 3. Mobil Haberleşme Nesilleri
- 4. Veri Dağıtım Şekilleri ve Akış Uygulamaları
- 5. Web Teknolojileri
- 6. Binary Search: Koşul ve Sezgi
- 7. Binary Search Akış Diyagramının Kurulması
- 8. Algoritmanın Analizi
- 9. first <= last Yerine first < last Yazılabilir mi?
- 10. Karmaşıklık ve Big O Gösterimi
- 11. Koşulların Sırası ve Sınır Kontrolü

1. IP Adresleri ve Alan Adları

Dönemin son dersinde hoca, ağ konusunu kaldığı yerden sürdürdü ve dersi bir algoritmayla bitireceğini söyledi. Bir önceki derste IP adreslerine kadar gelinmişti.

1.1 IPv4 ve IPv6

IP adresleri 32 bitten, yani 4 byte bilgiden oluşur. Bugün 128 bitlik adres kullanan IP version 6'ya geçmek için çalışmalar sürmektedir. Geçişin kısmen yapıldığı yerler de vardır. Geçişin sebebi, dünyadaki bütün cihazların benzersiz (unique) bir IP adresiyle birbirine erişebilmesini sağlamaktır. Özellikle IoT cihazlarının sayısının artması, IPv6 kullanımını artık zorunlu hâle getirmiştir.

1.2 Alan Adları

İnternette genel olarak IP adresleri üzerinden erişmeyiz. Bunun yapıldığı durumlar da vardır, ama sayısal adresleri ezbere bilmek ve akılda tutmak pek olası değildir. Bu yüzden isimler, yani alan adları (domain name) kullanılır. Alan adlarının altında alt alan adları (subdomain) bulunur. Üst düzey alan adlarına (top level domain) `edu`, `org`, `net` ve `com` örnek verilebilir. Bunların sonuna ülke kodları da eklenebilir. Hoca `com.tr`, `com.us` ve `co.uk` örneklerini verdi. Böylece ülkelere özel alan adları oluşur.

> **Ders sonrası not:** Hoca `com.us` örneğini de verdi; büyük olasılıkla ABD için yaygın kullanım doğrudan `.us` uzantısıdır. Ayrıca `com.tr` gibi bir adda en sağdaki `tr` büyük olasılıkla asıl üst düzey alan adıdır, `com` ise onun altındaki düzeydir.

1.3 Adreslerin Yönetimi

IP adreslerini yöneten yapı Internet Corporation for Assigned Names and Numbers'tır. Kâr amacı gütmeyen (non-profit) bu yapı, belirli adres bloklarını belirli kuruluşlara sağlar. Hocanın örneği kendi üniversitesiydi: Üniversitenin IP adreslerine bakarsanız belli blokların kullanıldığını görürsünüz, çünkü bu bloklar üniversiteye tahsis edilmiştir.

> **Ek bilgi (derste anlatılmadı):** Bu kuruluş büyük olasılıkla kısaca ICANN diye anılan kuruluştur. Blokların kuruluşlara ulaşması da büyük olasılıkla doğrudan değil, bölgesel internet kayıt kuruluşları ve servis sağlayıcılar üzerinden kademeli olarak gerçekleşir.

2. İnternet Uygulamaları ve Protokoller

Alan adı ve IP adresinden sonra hoca internet uygulamalarına (internet applications) geçti. Farklı türde uygulamalar vardır ve her biri bir protokol içerir. Haberleşme bu protokole uygun şekilde gerçekleşir.

> **Hocanın vurgusu:** Bu önemlidir. Örneğin FTP ile SMTP'nin kuralları (rule) tamamen birbirinden farklıdır, her protokol kendi kurallarına göre tasarlanmıştır.

2.1 FTP, Telnet ve SSH

File Transfer Protocol (FTP), adından da anlaşılacağı gibi dosya aktarımı için kullanılır. Telnet ve hocanın Telnet'in güvenli versiyonu olarak andığı Secure Shell (SSH) ise bir bilgisayardan başka bir bilgisayara, herhangi bir pencere (window) olmadan, komut satırıyla erişip o bilgisayar üzerinde işlem yapmayı sağlar. Yani bu protokollerle başka bilgisayarları kontrol edebilirsiniz.

> **Ders sonrası not:** Hoca SSH'yi Telnet'in güvenli versiyonu diye tanıttı; büyük olasılıkla SSH, Telnet'in bir sürümü değil, aynı işi şifreli yapan ayrı bir protokoldür.

2.2 HTTP ve HTTPS

Hyper Text Transfer Protocol (HTTP), bütün web sayfalarındaki bilgilerin aktarıldığı protokoldür. Güvenli versiyonu HTTPS diye geçer.

2.3 E-posta Protokolleri

Elektronik posta (e-mail) için farklı protokoller vardır. Simple Mail Transfer Protocol (SMTP) sayesinde mailleriniz mail sunucuları (mail server) arasında aktarılır. SMTP normalde metin mesajlarını ASCII koduyla kodlayarak gönderir.

ASCII olmayan (non-ASCII) veriyi de gönderebilmek için MIME denen başka bir protokol vardır. Açılımı Multipurpose Internet Mail Extensions'tır. Hoca bunun ileride duyabileceğiniz formatlardan biri olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** MIME büyük olasılıkla ayrı bir aktarım protokolünden çok, SMTP'nin taşıdığı iletinin biçimini genişleten bir standarttır. Dosya ekleri ve ASCII dışındaki karakterler büyük olasılıkla bu biçim sayesinde iletilebilir.

Maillere ulaşmak için ise iki yaklaşım vardır:

- **POP3 (Post Office Protocol, version 3):** Bu protokol kullanıldığında mesajlar bilgisayarınıza indirilir.
- **IMAP:** Bu protokol kullanıldığında mesajlar sunucuda tutulur ve her yerden erişilebilir. Son dönemde çok daha yaygın kullanılmaktadır.

Bu iki protokol, bir mail istemcisiyle (mail client) bağlandığınızda maillerin sunucu tarafında mı kalacağını, yoksa bilgisayarınıza mı indirileceğini belirler.

> **Ders sonrası not:** Hoca IMAP'in açılımını "Internet Mail Access Protocol" diye söyledi; bugünkü açılımı büyük olasılıkla "Internet Message Access Protocol"dür.

2.4 VoIP

Voice over Internet Protocol (VoIP), sesin veri paketleriyle taşınmasını sağlayan protokoldür. VoIP telefonlarla iki farklı noktada haberleşirken ses bilginizin tamamı veri paketleri üzerinden taşınır. Yani ses, normal telefon hattındaki ses bandı üzerinden değil, veri bandı üzerinden aktarılır.

3. Mobil Haberleşme Nesilleri

Hoca sonra mobil telefonların hikâyesine değindi. Yaklaşık 1980'lerden itibaren kullanılmaya başlayan mobil telefonlar 1G, 2G, 3G ve 4G'den geçerek bugüne geldi. 5G de yoldadır ve bazı yerlerde kullanılmaya başlanmıştır. Buradaki kısaltmalar first generation, second generation gibi nesil adlarının kısa hâlleridir.

- **1G:** Analog ses bilgisini havadan taşıyan ilk telefonlardır. Tek amaç, sesli haberleşmeyi kablosuz yapmaktı.
- **2G:** Sesler dijital sinyal olarak kodlanır ve havadan daha verimli taşınır. Buna ek olarak mesaj gibi bazı dijital veriler de telefonlar arasında gönderilmeye başlanır.
- **3G:** Önemli fark, yüksek veri hızlarına geçiştir. Görüntülü görüşme ve daha yüksek bant genişliği isteyen işler (televizyon veya video izlemek, video ve büyük resim göndermek) yapılmaya başlanır. Resim 2G ile de gönderilebiliyordu, ama daha kısıtlı miktarda.
- **4G:** Çok yüksek hızlarda erişim sağlanır. Aynı zamanda paket anahtarlamalı IP ağı (packet switched IP network) denen yapıya geçilmiştir. Hoca bunun ayrıntısına girmedi, ileride öğrenileceğini söyledi.

4. Veri Dağıtım Şekilleri ve Akış Uygulamaları

Hoca son dönemde ortaya çıkan bir kavramı sordu: on demand streaming applications.

Soru: Bu uygulamalar nedir? **Cevap:** Bir öğrenci Netflix ve YouTube'u örnek verdi: Kişi istediği zaman izleyebiliyor. Hoca bu cevabı onayladı.

Hoca konuya önce verinin dağıtılma şekilleriyle girdi:

- **Unicast:** Veri bir kullanıcıdan başka bir kullanıcıya gönderilir.
- **N-unicast:** Veri birden fazla kullanıcıya gönderilmek istendiğinde kullanılır.
- **Multicast:** N-unicast'in alternatifidir. Veri belirli bir gruba gönderilir.
- **Broadcast:** "Ben yayın yapayım, herkese gitsin" mantığıdır. Paket ağdaki herkese gönderilir, isteyen alır.

Bunların ayrıntısı ileride ağ dersinde görülecektir. Hocanın asıl varmak istediği nokta şuydu: Son dönemde olay, kullanıcının ilgili medyayı istediği zamanda ve istediği noktadan izlediği bir yapıya dönüştü. Bütün düzen buna göre kuruldu ve şu anda çok ciddi bir veri akışı gerçekleşiyor.

Hoca bu içeriğin okunmasını önerdi. Burada amaç daha çok ses ve video aktarımıdır. Konu şu sebeple çok daha önemli hâle geldi: Daha önce bahsedilen verilerin boyutu görece küçüktür. YouTube ve Netflix gibi multimedya içeriği sağlayan uygulamalar ise bant genişliğinde (bandwidth) çok ciddi yer kaplar.

Bu uygulamalar yüksek yoğunlukta veri aktarmalarına rağmen kesintisizdir ve diğer sitelere göre daha kaliteli görüntü sunar. Hocaya göre bunun en önemli sebeplerinden biri, internet servis sağlayıcılarıyla yaptıkları anlaşmalardır. Bu anlaşmalarla bant genişliğinin bir kısmını kendilerine ayrılmış (dedicated) olarak kullanırlar. Böylece başka bir siteye erişiminiz yavaşken bile buradaki görüntüler yüksek kaliteyle size ulaşabilir.

> **Ek bilgi (derste anlatılmadı):** Bu şirketler büyük olasılıkla içeriği kullanıcıya yakın sunuculara da yerleştirir. Bu yapıya büyük olasılıkla içerik dağıtım ağı (content delivery network, CDN) denir. Veri böylece daha kısa yol izleyerek geldiği için akış büyük olasılıkla daha az kesintiye uğrar.

5. Web Teknolojileri

5.1 Tarayıcı, Web Sunucusu ve URL

Tarayıcılar (browser), istediğiniz bilginin bilgisayarda belirli bir formatta gösterilmesini sağlar. Tarayıcı üzerinden dünyadaki birçok sunucuya erişirsiniz. Bilgilerin barındırıldığı (host edildiği) yer web sunucusudur (web server). Bilgiyi web sunucusundan talep edersiniz ve bilgi HTTP protokolüyle bilgisayarınıza gelir. Erişmeye çalıştığınız her belgenin webde bir adresi vardır. Buna URL (Uniform Resource Locator) denir.

5.2 HTML

Soru: HTML nedir, ne amaçla kullanılır? **Cevap:** Bir öğrenci açılımını Hypertext Markup Language diye verdi. Bir başka cevapta da verilerin tarayıcı tarafından anlaşılıp görsel olarak düzenlenmesini sağladığı söylendi. Hoca bunu şöyle toparladı: HTML, web sunucusundan tarayıcıya gönderilen ve istediğiniz bilginin belirli bir formatta gösterilmesi için kullanılan dildir.

HTML etiketlerle (tag) yazılır. `head` ve `body` gibi bölümler vardır. Bunların içinde çeşitli tanımlamalar yaparak tablo tanımlayabilir, paragraf açabilirsiniz. Böylece bilginiz, bugün eriştiğiniz internet sitelerindeki gibi formatlı biçimde gösterilir. Yeni versiyonu HTML5'tir ve HTML'e göre çok daha fazla yeteneğe sahiptir.

İşleyiş şöyledir: Sayfalar HTML kodlanarak hazırlanır. Siz istemci (client) tarafından sunucudan bu sayfaları istediğinizde, HTML kodları tarayıcı tarafından yorumlanır (interpret). Bu sayede iç içe tabloları, paragrafları, büyük harfle yazılmış veya renklendirilmiş metinleri ekranda görürsünüz. Sayfalar tarayıcıda yorumlandığı için aynı sayfa farklı tarayıcılarda bazen farklı görünebilir.

5.3 CSS

Görselleştirme için CSS'ten faydalanılır. HTML'in temel görevi sayfanın yapısını belirlemektir: tablolar ve sayfadaki bilgilerin nerede, nasıl konumlanacağı. Hocanın benzetmesiyle CSS buna yalnızca makyaj yapar. Ama HTML olmadan CSS'in pek bir anlamı yoktur, önce HTML ile "vücudu" oluşturmanız gerekir.

Hoca, şimdilik böyle düşünülebileceğini, ayrıntının zamanı geldiğinde sistem programlama dersinde ya da konuya ilgi duyulduğunda çok daha iyi öğrenileceğini söyledi.

5.4 Dinamik Sayfalar

Yalnız HTML ile statik sayfalar oluşturulur. Kullanıcıyla etkileşimli (dinamik) sayfalar istiyorsanız başka programlama dillerini de kullanmanız gerekir. Hoca JavaScript ve PHP'yi örnek verdi. HTML5 bu konuda daha farklı imkânlar da sunar.

5.5 XML ve JSON

Extensible Markup Language (XML), verilerin temsil edilmesi ve bir noktadan başka bir noktaya taşınması için kullanılan dildir. JSON onun rakibi, yani muadilidir. İkisinin birbirine göre artı ve eksi yönleri vardır.

5.6 Ağ Katmanları

Hoca son olarak bir öğrencinin daha önce paylaştığı katmanları andı: Application Layer, Transport Layer, Network Layer, Data Link Layer ve Physical Layer. Bu katmanların ayrıntısına girmedi, çünkü bu başka bir dersin konusudur. Bu bölümde geçenlerin, genel olarak bilinmesi gereken başlıklar olduğunu ve bunlara kabaca bir kez daha göz atmanın faydalı olacağını söyledi.

6. Binary Search: Koşul ve Sezgi

6.1 Doğrusal Aramadan Binary Search'e

Dizide arama yapmayı daha önce görmüştük. Buna sequential search ya da linear search demiştik. Bu aramanın sıralı ve sırasız dizideki hâli biraz farklıydı, sıralı dizide biraz daha etkili bir algoritma yazılabiliyordu. Ama en kötü durum senaryosu ikisinde de eleman sayısı kadar, yani n karşılaştırma yapmak demekti.

> **Hocanın vurgusu:** Binary search algoritmasının olmazsa olmaz koşulu, arama yapılacak dizinin sıralı olmasıdır. Sırasız dizide bu algoritma uygulanamaz.

6.2 Algoritmanın Mantığı

Soru: Binary search nasıl çalışır? **Cevap:** Bir öğrenci şöyle özetledi: Önce dizinin ortasındaki eleman bulunur ve aranan elemanla karşılaştırılır. Aranan eleman ondan büyükse doğrudan sağ tarafa geçilir, sol taraf atlanır. Sağ tarafta yine ortadaki eleman bulunup karşılaştırılır. Bu böyle sürer ve sonunda istenen elemana ulaşılır. Hoca bu açıklamayı onayladı.

6.3 Sayı Tahmin Oyunu

Hocanın sezgi için verdiği örnek bir oyundur. 0 ile 100 arasında bir sayı tuttuğunuzu düşünün, diyelim ki 60. Karşı taraf her seferinde bir tahmin yapar ve tuttuğunuz sayının bu tahminden büyük mü küçük mü olduğunu öğrenir. Kişinin stratejisi genelde şöyle olur:

1. Önce ortadaki sayı sorulur: 50. Cevap "büyük" olur.
2. Sonra 50 ile 100'ün ortası sorulur: 75. Cevap "küçük" olur.
3. 50 ile 75'in ortası $(50 + 75)/2 = 62{,}5$ eder, diyelim 62 sorulur. Cevap "küçük" olur.
4. Artık 50 ile 62 arasında aranır: $(50+62)/2 = 56$. 60 bundan büyüktür (ara adım eklendi).
5. 56 ile 62 arasında aranır: $(56 + 62)/2 = 59$. 60 bundan da büyüktür (ara adım eklendi).
6. 59 ile 62 arasında aranır: $(59 + 62)/2 = 60{,}5$, yani 60 sorulur ve sayı bulunur (ara adım eklendi).

Hoca, hissiyatla tahmin etmeye çalışanlar olsa da bilimsel olarak sayının en hızlı bu şekilde tahmin edildiğini söyledi. Binary search de aynı mantığı kullanır.

6.4 Dizi Üzerinde İlk Örnek

Hoca akış diyagramı çizileceği için dizinin gözlerini 0'dan 5'e değil, 1'den 6'ya kadar numaraladı. İkisinin de kullanılabileceğini, fark etmeyeceğini söyledi. Dizideki değerler 1, 2, 3, 4, 5, 7'dir ve aranan eleman 4'tür.

İlk tahmin orta noktayla yapılır. 4, 3'ten büyük olduğu için ikinci tahmin sağ tarafta yapılır. Orta noktayla son noktanın ortası bulunur. 4 oradaki değerden küçük olduğu için bu kez o iki nokta arasında tahmin yapılır ve sonunda 4 bulunur. Önemli olan, her turda hangi tarafa gidileceğine karar vermek, gidilen tarafta orta noktaya bakmak ve bulunmadığı sürece devam etmektir. Bunun yanında özellikle belirlenmesi gereken bir durma koşulu vardır.

6.5 Ders İçi Problem

Hoca bu noktada öğrencilere süre verdi ve internete bakmadan şu problemi çözmelerini istedi:

> Verilen bir dizide aranan bir elemanın olup olmadığını, varsa nerede olduğunu binary search yaklaşımıyla bulan algoritmanın akış diyagramını çiziniz.

İpucu olarak dizinin başlangıç ve son adresiyle başlamayı, orta noktaya bakmayı, orta noktadan büyükse sağa, küçükse sola gitmeyi söyledi. Dizi sıralı kabul edilir, sıralamayla uğraşılmaz.

7. Binary Search Akış Diyagramının Kurulması

7.1 Bir Öğrencinin Çözümü

Bir öğrenci çözümünü do-while ile anlattı. Diziyi ve eleman sayısını aldıktan sonra `start` değişkenine 0, `end` değişkenine eleman sayısının bir eksiğini atamıştı. Döngünün içinde `mid` değişkenini `start` ile `end`'in toplamının yarısı olarak hesaplıyordu. Dizinin `mid` gözündeki değer aranan değerden küçükse `start`'ı `mid + 1`, büyükse `end`'i `mid - 1` yapıyordu. Döngü koşulu ise o gözdeki değerin x'e eşit olmamasıydı. Hoca bu çözümün genel olarak doğru göründüğünü söyledi ve kendi çözümüne geçti.

> **Ders sonrası not:** Hoca bu çözüm için "genel olarak doğru" dedi. Python denemesine göre bu sürüm, aranan eleman dizide yoksa bitmez. Dersteki `1 2 3 5 7 8` dizisinde 4 arandığında `start = 3`, `end = 2`, `mid = 2` değerlerinde takılır. Hoca büyük olasılıkla genel akışı kastetti. Eksik olan ikinci koşul aşağıda dersin içinde tamamlanmaktadır.

7.2 Girdiler ve İlk Atamalar

Dizi $A[N]$ ve aranan eleman X olsun. Yani dışarıdan üç girdi alınır: N, A ve X. Hoca ilk gözü 1 kabul ettiği için başlangıç atamaları şöyledir:

- `first = 1`
- `last = N`
- `middle = (first + last) DIV 2`

Hoca, öğrencinin do-while kullanmasının yanlış olmadığını, kendisinin işi baştan yapıp while ile kontrol edeceğini söyledi.

7.3 Döngü Koşulu

Soru: While'da hangi kontrol yazılmalı? **Cevap:** Bir öğrenci `middle`'ın X'ten büyük olmasına bakmayı önerdi. Hoca bunun çok etkili bir çözüm olmadığını söyledi. Aranan koşul, $A[middle]$'ın X'ten farklı olmasıdır. Eşit olmadığı sürece döngü devam eder.

Hoca başka bir koşula ihtiyaç olup olmadığını da sordu, ama cevap gelmeyince buna diyagramın sonunda döndü.

7.4 Döngü İçindeki Karar

Soru: Eşit olmadığı sürece ne yapılacak? **Cevap:** Bir öğrenci $A[middle]$'ın X'ten küçük olup olmadığına bakmayı önerdi. Hoca, karışıklık olmasın diye bunu "X, $A[middle]$'dan büyük mü?" biçiminde yazdı.

Koşul doğruysa aranan eleman sağ taraftadır. Bu durumda `first = middle + 1` yapılır. Bir öğrencinin belirttiği gibi `last`'ta bir değişiklik olmaz.

> **Hocanın vurgusu:** `first`'e `middle` değil `middle + 1` atanmalıdır. Yoksa aynı yerde takılıp kalmaya sebep olan bir yapı oluşur. Aramaya, bakılan yerin bir sonraki gözünden başlamak gerekir.

Soru: Koşul yanlışsa ne yapılacak? **Cevap:** Bir öğrenci doğrudan `middle = middle / 2` yazdığını söyledi. Hoca böyle olmaması gerektiğini söyledi. Küçük tarafta, örneğin 2'yi aradığımızı düşünelim. Önce orta noktaya baktık, şimdi onun solundaki aralığa bakacağız. O zaman `last` değişmelidir. Hoca önceki çizimde burayı 3 olarak göstermişti, ama doğrusu `last = middle - 1`'dir, yani bir küçük gözden başlanır.

Soru: Bu güncellemelerden sonra ne yapılacak? **Cevap:** Bir öğrencinin dediği gibi yeni bir `middle` belirlenir: `middle = (first + last) DIV 2`. Ardından döngünün başına dönülür.

7.5 İkinci Koşul

Hoca, farklı olduğu sürece döneceğimizi, ama bir koşulun daha kontrol edilmesi gerektiğini hatırlattı.

Soru: Bu koşul ne olmalı? **Cevap:** Bir öğrenci `last` ile `first` eşit olmadığı sürece dönmeyi önerdi. Hoca bunun söylenebileceğini, ama `first`'ün `last`'tan küçük eşit olduğu sürece dönmenin aynı kapıya çıkıp çıkmayacağını veya daha uygun olup olmayacağını sordu. Birlikte denemeye karar verdiler. Sonuçta koşul `first <= last AND A[middle] ≠ X` oldu.

7.6 Döngüden Çıkış

Soru: Döngüden çıktıktan sonra ne yapılacak? **Cevap:** Bir öğrenci, hangi nedenle çıkıldığını öğrenmek için bir if kontrolü açmayı ve $A[middle]$'ın X'e eşit olup olmadığına bakmayı önerdi. Hoca bunu diyagrama ekledi. Eşitse elemanın kaçıncı sırada bulunduğu yazdırılır, değilse bulunamadığı (not found) yazdırılır.

7.7 Tam Sayı Bölmesi

Analiz sırasında bir öğrenci, akış diyagramlarında tam sayı bölmesi için hep `DIV` kullanıldığını hatırlattı. Hoca haklı olduğunu söyledi, hızlı çizerken atladığını belirtti ve iki `middle` hesabını da `DIV 2` olarak düzeltti.

7.8 Akış Diyagramının Son Hâli

Akış diyagramı adım adım şöyle okunur:

```
Oku: N, A[N], X
first = 1
last = N
middle = (first + last) DIV 2

first <= last VE A[middle] != X oldugu surece:
    X > A[middle] ise:
        first = middle + 1
    degilse:
        last = middle - 1
    middle = (first + last) DIV 2

A[middle] = X ise:
    Yaz: middle (bulunan sira)
degilse:
    Yaz: bulunamadi
```

8. Algoritmanın Analizi

Hoca algoritmanın doğruluğunu kontrol etmek için N, `first`, `last`, `middle`, X ve $A[middle]$ sütunlarıyla analiz yaptı. Burada iki örnek çözüldü: aranan elemanın dizide olduğu ve olmadığı durum.

8.1 Aranan Eleman Dizide Var

Verilen: $A = 1, 2, 3, 4, 5, 7$, $N = 6$, $X = 4$.

Adım	first	last	middle	A[middle]	Kontrol	Sonuç
1	1	6	$(1+6)$ DIV $2 = 3$	3	$3 \neq 4$, $4 > 3$	`first = 4`
2	4	6	$(4+6)$ DIV $2 = 5$	5	$5 \neq 4$, $4 > 5$ yanlış	`last = 4`
3	4	4	$(4+4)$ DIV $2 = 4$	4	$4 = 4$	Döngüden çıkılır

İlk adımda $7/2 = 3{,}5$ olur, tam sayı bölmesi bunu 3 yapar. Son adımda $A[middle]$, X'ten farklı olmadığı için döngüden çıkılır. Çıkıştaki kontrol doğrudur ve elemanın 4. sırada bulunduğu yazdırılır. Hoca bu örnek için bir sıkıntı görünmediğini söyledi.

8.2 Aranan Eleman Dizide Yok

Hoca dizinin ortasını değiştirerek ikinci durumu denedi.

Verilen: $A = 1, 2, 3, 5, 7, 8$, $N = 6$, $X = 4$.

Adım	first	last	middle	A[middle]	Kontrol	Sonuç
1	1	6	3	3	$3 \neq 4$, $4 > 3$	`first = 4`
2	4	6	5	7	$7 \neq 4$, $4 > 7$ yanlış	`last = 4`
3	4	4	4	5	$5 \neq 4$, $4 > 5$ yanlış	`last = 3`
4	4	3	$(4 + 3)$ DIV $2 = 3$ (ara adım eklendi)	3	$4 \leq 3$ yanlış	Döngüden çıkılır

Üçüncü adımda `first` ve `last` eşittir. Küçük eşit koşulu sayesinde kontrol devam eder ve $A[4]$ de incelenir. Dördüncü adımda hocanın belirttiği gibi `middle`'ın kaç olduğunun artık pek önemi yoktur, çünkü `first <= last` yanlıştır. İlk koşul sağlanmadığı için ikinci koşula bakılmaz. Döngüden çıkılır. Çıkışta $A[middle]$, X'e eşit olmadığı için "bulunamadı" yazdırılır. Yani algoritma bu durumda da çalışır.

9. first <= last Yerine first < last Yazılabilir mi?

Bir öğrenci, ikinci koşulu kullanmadan yalnız $A[middle] \neq X$ koşuluyla döngüyü kurup döngü içindeki if'te ek bir kol açmayı düşündüğünü anlattı. Ardından tartışma `first <= last` koşulundaki eşittirin gerekli olup olmadığına döndü.

Soru: Bu koşulda yalnız küçüktür koysak yeterli olur mu? **Cevap:** Üç öğrenci yeterli olacağını söyledi. Birincisi, aranan eleman dizide yoksa gereksiz bir turun atlanacağını ve daha hızlı bitileceğini belirtti. İkincisi, `first` ve `last` eşitse `middle`'ın da onlara eşit olacağını, zaten bu değer kontrol edildiği için eşitlik durumuna bakmaya gerek olmadığını söyledi. Üçüncüsü, döngüye bir kez az girileceğini ekledi.

Hoca üçünün de aynı noktaya değindiğini ve doğru göründüğünü söyledi. Çıkışta $A[middle]$'ın X'e eşit olup olmadığına bakıldığı için eşittiri çıkarmak çalışır görünür. Eşittir gereksiz bir tur daha yaratabileceğinden, kaldırmak akıllıca görünür. Fakat bu sonuç çıkıştaki kontrole bağlıdır. Çıkışta `first`'ün `last`'tan büyük, büyük eşit ya da küçük eşit olup olmadığı sorulsaydı, belki bu eşittire ihtiyaç olurdu. Hoca bu izlemeyi öğrencilere bıraktı.

> **Ders sonrası not:** Hoca "çalışır gibi gözüküyor" deyip izlemeyi öğrencilere bıraktı. Akış diyagramı python ile iki koşulla da denendi: 1-8 elemanlı rastgele sıralı dizilerde, dizide olan ve olmayan değerlerle 2.016.000 aramada hata çıkmadı. İkinci örnekte `first < last` ile döngü `first = last = middle = 4` olunca biter ve $A[4] = 5 \neq 4$ olduğu için "bulunamadı" yazılır. Bu yüzden hocanın değerlendirmesi büyük olasılıkla genel olarak doğrudur.

10. Karmaşıklık ve Big O Gösterimi

Sequential search en kötü durumda n adımda tamamlanıyordu, yani karmaşıklığı $O(n)$'di. Binary search ise $\log n$ adım alır. Bunu 2'nin kuvveti olan eleman sayılarıyla düşünebilirsiniz, $\log_2 n$ olarak:

- 8 eleman varsa $\log_2 8 = 3$ adımda sonuca ulaşılır.
- 1024 eleman varsa, 1024 kontrol yerine 10 kontrolle sonuca ulaşılır.

  Ders sonrası not: Hoca 8 elemanda 3, 1024 elemanda 10 adım dedi. Akış diyagramı python ile denendiğinde en kötü durumda bakılan `middle` sayısı 8 elemanda 4, 1024 elemanda 11 çıktı. Hoca büyük olasılıkla yaklaşık değer olan $\log_2 n$'i kastetti.

  Hocanın vurgusu: Sıralamak, sıralı dizilerle çalışmak ve binary search kullanarak $\log n$ karmaşıklığında algoritmalar tasarlamak oldukça önemlidir, çünkü verimliliği çok artırır. n ile $\log n$ arasında ciddi bir fark vardır: Birinde 1024, öbüründe 10. Bu makas gittikçe açılır.

Big O, bir algoritmanın karmaşıklığını ifade ederken kullandığımız önemli bir gösterimdir. Genelde girdi sayısı cinsinden yazılır: $O(n^2)$, $O(n^3)$, $O(n \log n)$, $O(1)$ gibi. Karmaşıklık önemlidir, çünkü algoritmanın ne kadar verimli olduğunu gösterir. Hoca, bunlara algoritma analizi ve veri yapıları derslerinde daha aşina olunacağını söyledi. Binary search de ileride yeniden görülecektir, burada yalnız giriş yapılmıştır.

11. Koşulların Sırası ve Sınır Kontrolü

Soru: While'da iki koşul varsa ve arasında VE operatörü varsa, birinci koşul sağlanmadan bilgisayar ikinci koşulu kontrol eder mi? **Cevap:** Etmez. Birinci kontrol başarısız olursa ikinci kontrol yapılmaz. VEYA varsa ise tabii ki yapılır.

> **Hocanın vurgusu:** Burası kritiktir. Hangi kontrolün önce yapılacağını buna göre seçmelisiniz. Dizilerde önce o indise erişme hakkınız olup olmadığını kontrol etmeli, ondan sonra oradaki değere bakmalısınız. Önce değere erişmeye çalışırsanız bazen erişmemeniz gereken bir yere erişirsiniz. Bu yüzden sınır kontrolleri her zaman önce yapılmalıdır.
>
> **Ek bilgi (derste anlatılmadı):** Bu davranışa büyük olasılıkla kısa devre değerlendirmesi (short-circuit evaluation) denir. C'de `&&` büyük olasılıkla birinci koşul yanlışsa, `||` ise birinci koşul doğruysa ikinci koşula bakmaz.
>
> **Ders sonrası not:** Hocanın kuralı akış diyagramındaki çıkış kontrolü için de geçerlidir. Aranan değer bütün elemanlardan küçükse döngü `first = 1`, `last = 0`, `middle = 0` ile biter ve çıkıştaki $A[middle] = X$ kontrolü dizide bulunmayan $A[0]$'a bakar. Python denemesinde bu durum, değerin dizinin en küçük elemanından küçük olduğu aramalarda ortaya çıktı. Büyük olasılıkla daha güvenli yol, döngüden sonra önce `first <= last` koşuluna bakmaktır. Aşağıdaki C programı akış diyagramının 0'dan başlayan indislerle yazılmış karşılığıdır ve bunu yapar. Program `gcc -Wall` ile uyarısız derlendi. Dersteki iki dizi için 3 ve -1 yazdırır: 3, hocanın 1'den başlayan numaralandırmasındaki 4. sıraya karşılık gelir, -1 ise bulunamadı demektir.

```c
#include <stdio.h>

/* Akis diyagraminin C karsiligi: indisler 0'dan baslar.
   Bulunursa indis, bulunmazsa -1 dondurur. */
int ikili_arama(const int A[], int N, int X)
{
    int first = 0, last = N - 1;
    int middle = (first + last) / 2;

    while (first <= last && A[middle] != X) {
        if (X > A[middle])
            first = middle + 1;
        else
            last = middle - 1;
        middle = (first + last) / 2;
    }
    if (first <= last)
        return middle;
    return -1;
```

```
}

int main(void)
{
    int A1[] = {1, 2, 3, 4, 5, 7};
    int A2[] = {1, 2, 3, 5, 7, 8};

    printf("%d\n", ikili_arama(A1, 6, 4));
    printf("%d\n", ikili_arama(A2, 6, 4));
    return 0;
}
```

Dersin Toparlaması

Bu derste ağ konusunun kalan başlıkları tamamlandı. IP adreslerinden ve alan adlarından başlanarak FTP, Telnet, SSH, HTTP, SMTP, MIME, POP3, IMAP ve VoIP gibi uygulama protokolleri, mobil haberleşme nesilleri, veri dağıtım şekilleri ve akış uygulamaları görüldü. Web tarafında HTML'in yapıyı, CSS'in görünümü belirlediği, dinamiklik için JavaScript ve PHP gibi dillere ihtiyaç duyulduğu, XML ve JSON'ın ise veriyi temsil edip taşıdığı anlatıldı. Hoca bu başlıkların ayrıntısının ilerideki ağ derslerinde öğrenileceğini söyledi.

İkinci bölümde binary search algoritması akış diyagramıyla kuruldu ve iki örnekle analiz edildi. Temel fikir, sıralı dizide her turda ortadaki elemana bakıp arama aralığını daraltmaktır. Bunun için `first = middle + 1` ve `last = middle - 1` güncellemeleri, her turda `middle`'ın yeniden hesaplanması ve bir durma koşulu gerekir. Binary search'ün log n karmaşıklığı, algoritma analizi ve veri yapıları derslerine bir hazırlıktır.

Hoca kapanışta, ikinci sınıftan itibaren hocaların hangi konularda çalıştığına bakmayı ve ilgi duyulan alanda araştırma gruplarına katılmak için hocalarla görüşmeyi tavsiye etti. Ara ve bitirme projelerinde de insanı geliştirecek ve özgeçmişte iyi duracak konular seçilmesini önerdi.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.